Informe de validación integral del software

Sistema de inventario Bodega Sneakers · Épica 8 (Sprint INV 7) · 12 de noviembre de 2026

Proyecto SCRUM (Jira) · Repositorio: https://github.com/Maiguell/bodegaplus (rama main)

Resumen. Se ejecutaron 30 pruebas unitarias (30 aprobadas), 9 pruebas de integración de la API (9 aprobadas) y 84 verificaciones de criterios de aceptación de extremo a extremo (84 aprobadas). Con 50 usuarios simultáneos durante 4 minutos el sistema atendió 5.600 peticiones sin errores, con un percentil 95 de 300 ms. La prueba de estrés en rampa llegó hasta 400 usuarios. **La prueba de usabilidad con cliente real está pendiente** (sección 3).

1. Pruebas funcionales (25 %)

1.1 Pruebas unitarias (HT09) — node:test

30/30	100 %	86.8 %	95.6 %
pruebas aprobadas	cobertura de líneas	cobertura de ramas	cobertura de funciones

Grupo	Casos (éxito y límite)	Resultado
HU01 Generación de variantes y SKU	14 variantes de 7 tallas × 2 colores; ids repetidos; formato de SKU	**3/3** ✓
HT05 Código de barras interno	EAN-13 con prefijo 20 y dígito de control; unicidad; ids inválidos	**3/3** ✓
HT04 Cálculo de saldo y stock negativo	suma/resta; bloqueo de saldo negativo; cantidades no enteras; saldo acumulado del kardex	**4/4** ✓
HU11 Causales de ajuste	causal activa; vacía, inexistente o inactiva	**2/2** ✓
HU14 Expiración de reservas	vencimiento a las 48 h; vigente / vencida en el límite	**2/2** ✓
HU15 Alerta y sugerido de reposición	en el mínimo no alerta; por debajo sí; sugerido nunca negativo	**2/2** ✓
HU16 Cálculo de rotación	existencia promedio; exclusión de días sin kardex; índice y división por cero	**3/3** ✓
HU17 Valorización	agrupación por sede; total por categoría = total general	**2/2** ✓
RNF-05 / HU20 Contraseñas y bloqueo	política (8+, letras y números); bloqueo de 15 min al 5.º intento	**3/3** ✓
HU02 Plantilla CSV del catálogo	separadores y comillas; rangos de tallas; motivos de rechazo	**3/3** ✓
HU19 Permisos por rol y sede	admin/auditor; vendedor sin aprobar; 403 en otra sede	**3/3** ✓

Cobertura medida sobre los módulos de reglas de negocio (lib/reglas.js, lib/permisos.js, utils/sku.js). Comando: `npm run test:cobertura`; reporte LCOV en backend/coverage/lcov.info.

1.2 Pruebas de integración de la API (HT10) — Supertest + MySQL

#	Prueba	Valida	Resultado
1	Autenticación JWT	200 con token; 401 sin token; 403 sin permiso; renovación; bloqueo 423 al 5.º intento registrado en BD	✓ 726 ms
2	Entrada contra OC hasta el kardex	201, OC Recibida parcial, stock +N y fila ENTRADA en kardex con la OC; exceso sin observación → 400	✓ 35 ms
3	Venta que descuenta stock	201, stock −1, movimiento VENTA; vendedor en otra sede → 403; sin stock → 409 con sugerencias y sin mover stock	✓ 34 ms
4	Traslado envío–recepción	En tránsito descuenta origen y no suma destino; recepción suma destino y cierra	✓ 35 ms
5	Disponibilidad multisede	matriz talla × sede igual a la BD y respuesta < 3 s	✓ 17 ms
6	Referencia con matriz (HU01)	14 variantes con SKU únicos y códigos EAN-13; 403 para vendedor	✓ 41 ms
7	Ajuste con aprobación	pendiente no mueve stock; vendedor no aprueba (403); aprobado descuenta	✓ 35 ms
8	Importación de catálogo CSV	1 referencia y 6 variantes creadas; filas 3 y 4 rechazadas con motivo	✓ 23 ms
9	Integridad del kardex	saldo calculado desde el kardex = stock almacenado	✓ 10 ms

Cada prueba verifica el código HTTP, el cuerpo de la respuesta y el efecto en la base de datos (stock, kardex, estados). Base aislada que se recrea en cada corrida. Comando: `npm run test:integracion`.

1.3 Criterios de aceptación de extremo a extremo

Suite	Historias	Verificaciones	Resultado
Sprints 3 y 4	HT04, HU05–HU10, HU13, HU14	45	45/45 ✓
Sprints 5 y 6	HU11, HU12, HU15–HU18, HT06–HT08	39	39/39 ✓

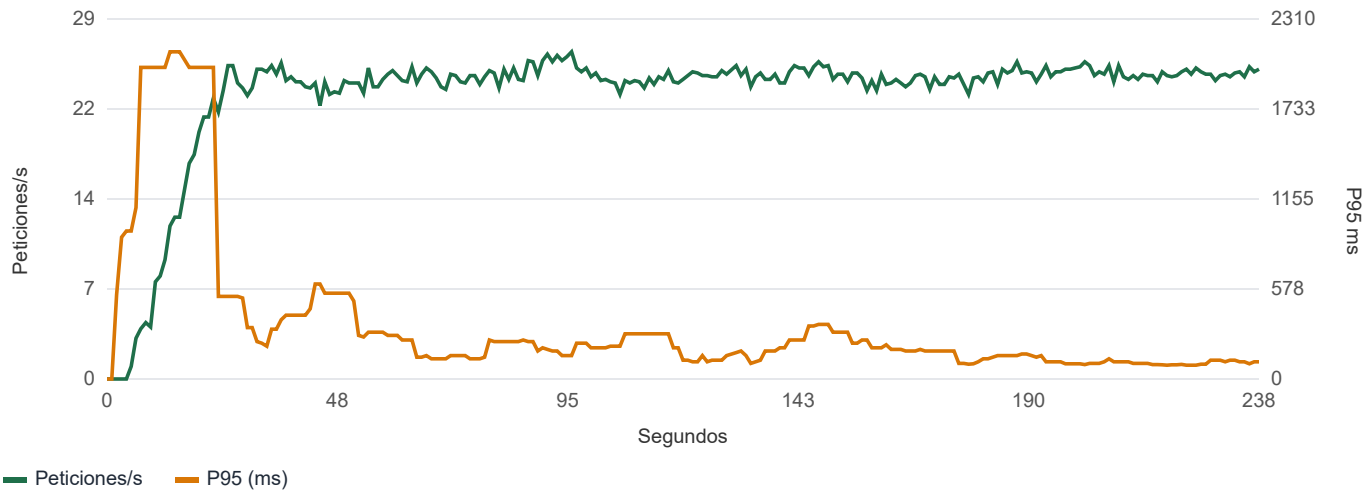
Incluye concurrencia (dos ventas simultáneas por la última unidad: solo una se confirma), inmutabilidad del kardex y restauración de respaldos (26 tablas y 6 triggers idénticos).

2. Pruebas de rendimiento y estrés (25 %)

2.1 Carga normal (HT11) — Locust, 50 usuarios simultáneos, 4 minutos

Flujos por usuario (vendedor): iniciar sesión (peso 1), consultar disponibilidad multisede (peso 6) y registrar venta (peso 3), con 1 a 3 s de pausa entre acciones.

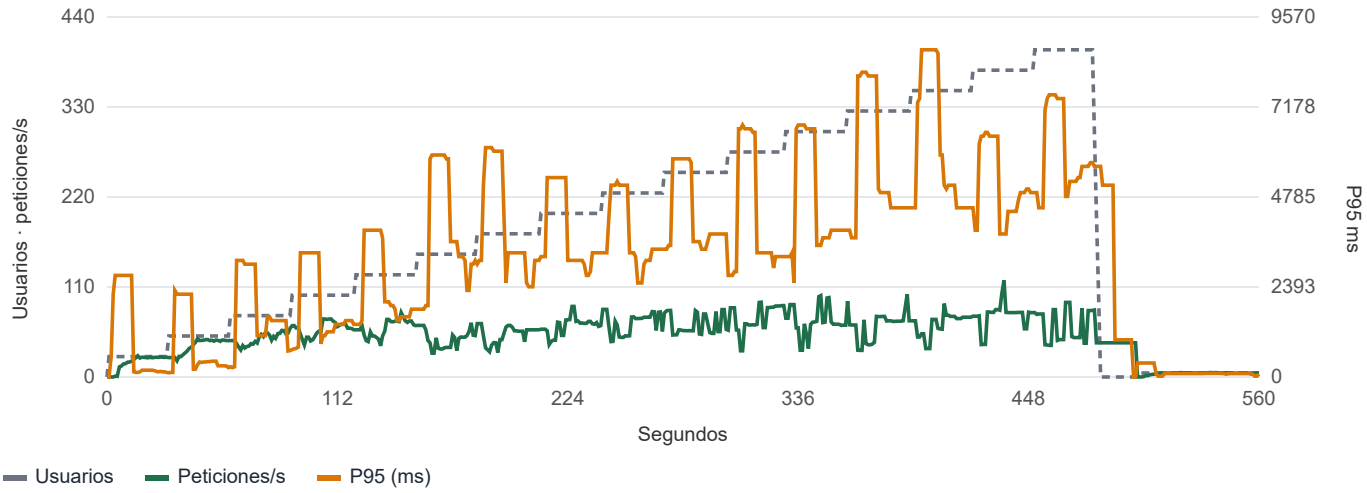
Operación	Peticiones	Fallos	Promedio (ms)	Mediana (ms)	P95 (ms)	Máx. (ms)	Peticiones/s
1. Iniciar sesi&#65533;n	589	0	248	130	890	1.984	2,5
2. Consultar disponibilidad	3.339	0	52	7	210	2.265	14
3. Registrar venta	1.672	0	64	16	240	2.376	7
Total	**5.600**	**0**	**76**	**10**	**300**	**2.376**	**23,5**



Interpretación. Con 50 vendedores simultáneos el sistema atendió todas las peticiones sin errores. La consulta de disponibilidad y la venta responden en decenas de milisegundos en la mediana; la operación más costosa es el inicio de sesión, porque el hash bcrypt es deliberadamente lento (protección contra ataques de fuerza bruta). Los picos de 2 s coinciden con varios inicios de sesión simultáneos.
RNF-02 (consulta multisede < 3 s): Cumple: la consulta multisede respondió en 51 ms de promedio y 210 ms en el percentil 95; ninguna superó 3 s.
Frontend (Google Lighthouse, escritorio): First Contentful Paint 0,5 s, Largest Contentful Paint 0,5 s, puntaje de rendimiento 100/100.

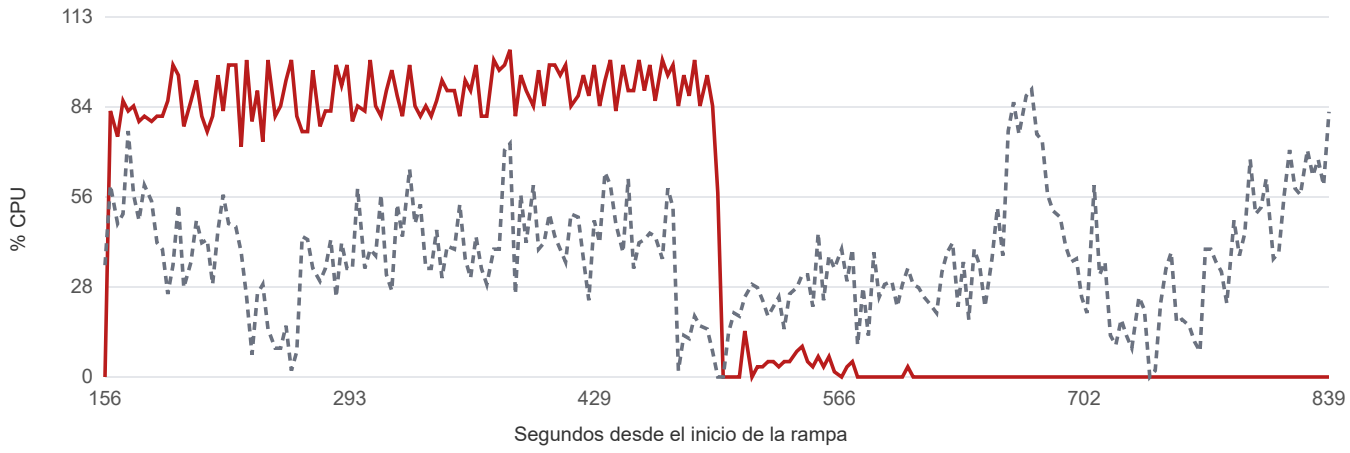
2.2 Estrés en rampa y punto de quiebre (HT12)

Escalones de +25 usuarios cada 30 s hasta 400, con 0,5 a 1,5 s de pausa; luego se baja la carga a 0 y se mantienen 5 usuarios durante 60 s para medir la recuperación.



Usuarios	Peticiones/s	Fallos/s	Mediana (ms)	P95 (ms)
5	4,4	0	14	126
25	18	0	148	1.022
50	37,5	0	28	880
75	46,6	0,43	242	1.776
100	58,8	1,37	590	1.955
125	60	2,07	919	2.417
150	46,5	6,59	2.286	3.810
175	52,2	5,13	2.203	3.997
200	62	7,89	2.077	3.894
225	66	9,53	2.420	3.783
250	64,2	18,72	2.923	4.350
275	72	10,64	2.739	4.225
300	68,9	20,06	3.323	4.642
325	62,4	31,46	4.547	5.640
350	64,8	30,76	4.293	6.157
375	75,5	37,73	3.870	5.090
400	62,6	34,21	4.931	5.938

Recursos durante el estrés



Node.js ejecuta el código en un solo hilo: 100 % de un núcleo es su techo. RAM del backend entre 197 y 212 MB. MySQL corre como servicio de Windows y sus contadores de CPU no son legibles sin privilegios de administrador, por eso no se grafica. Muestreo cada 2 s (backend/tests/carga/monitor.ps1); el monitor comenzó a los 156 s de la rampa.

Punto de degradación: El throughput deja de crecer hacia los 100 usuarios (≈ 60–70 peticiones/s): desde ahí más usuarios solo aumentan la espera. La degradación drástica ocurre alrededor de 150 usuarios concurrentes con pausas de 0,5–1,5 s: la mediana salta de 0,9 s a 2,3 s y aparecen consultas de disponibilidad de más de 3 s (RNF-02). Con 325 usuarios o más, cerca de la mitad de las consultas superan 3 s (máximo 8,8 s).
Componente que falla primero: La CPU del proceso Node.js: llega al 100 % de su único núcleo (máximo 102 %), mientras el equipo completo no pasa del 77 % y la RAM del backend se mantiene estable (≈ 200–212 MB, sin fugas). No hubo errores HTTP 5xx, ni bloqueos de base de datos, ni conexiones rechazadas: 8.583 ventas y 3.202 inicios de sesión sin fallos. El inicio de sesión (hash bcrypt) es la operación que más CPU consume.
Recuperación: Al bajar la carga a 5 usuarios, la mediana volvió a 5–7 ms en unos 10 segundos, sin errores y sin reiniciar el servicio. La verificación posterior del kardex no encontró diferencias en 480 saldos ni stock negativo: la concurrencia no corrompió datos.
Umbrales de tolerancia: Con una instancia: hasta ~75 usuarios activos simultáneos se cumple RNF-02 con holgura; entre 75 y 150 la respuesta es aceptable (p95 de 1,8 a 2,4 s); más de 150 requiere escalar horizontalmente (varias instancias) o el modo clúster de Node.

Entorno: una sola instancia de Node.js y MySQL 8 en el mismo equipo de desarrollo (Windows 11, 16 núcleos lógicos, Node.js 24, MySQL 8.0 local). En la nube los valores absolutos cambiarán; la forma de la curva y el cuello de botella son los relevantes.

3. Prueba de usabilidad con cliente real (40 %)

> **PENDIENTE — requiere la sesión con un cliente real.** Este apartado se completa con los datos de la sesión (HT13–HT15). El material está listo: guía, consentimiento, ficha técnica, guion, tres escenarios, matriz de observación y plantilla de hallazgos en `docs/usabilidad/guia-prueba-usabilidad.md`; los datos de los escenarios se cargan con `npm run escenarios`.

Elemento	Estado
Ficha técnica del participante (perfil, fecha, hora)	**Pendiente**
Consentimiento de grabación firmado	**Pendiente**
Enlace al video (Drive / YouTube oculto)	**Pendiente: _____**
Escenario 1 — Tenis Urbano 2045 talla 41 en qué sede	**Datos listos; sesión pendiente**
Escenario 2 — Recibir OC-0012 y dejar etiquetas listas	**Datos listos; sesión pendiente**
Escenario 3 — Cambio de botas 39 → 40 sin stock en la tienda	**Datos listos; sesión pendiente**
Matriz de observación por escenario	**Pendiente**
Hallazgos U-01, U-02… con severidad y solución	**Pendiente**

Evaluación heurística previa del equipo (no sustituye la sesión)

ID	Problema	Severidad	Corrección
H-01	Cierre del conteo: Aceptar/Cancelar decidía si lo no contado quedaba en 0 (generó un ajuste de −820 pares).	Alta	Modal con dos opciones explícitas; la segura por defecto.
H-02	Se podía registrar un ajuste imposible de aprobar (restar más de lo existente).	Media	Validación al registrar (pantalla y servidor).
H-03	El buscador del conteo no encontraba el texto copiado de la tabla.	Media	Búsqueda por palabras.
H-04	En celular el contenido se cortaba por los costados.	Alta	Columnas con minmax(0, 1fr) y tablas desplazables.
H-05	Rotación con existencia promedio subestimada antes de la apertura del kardex.	Media	Promedio desde la apertura, con nota.

4. Conclusiones y plan de acción priorizado (10 %)

Funcionalmente el sistema cumple los criterios de aceptación de las historias de los sprints 1 a 7: 30/30 pruebas unitarias, 9/9 de integración y 84/84 verificaciones de extremo a extremo, incluidas la concurrencia en ventas y la inmutabilidad del kardex.

En condiciones normales de trabajo (50 usuarios) el desempeño es holgado y se cumple RNF-02. La prueba de estrés muestra que una sola instancia de Node.js satura su núcleo de CPU y desde ese punto los tiempos crecen; el sistema no pierde datos ni se cae y se recupera al bajar la carga.

Pendiente lo más importante para el usuario final: la prueba de usabilidad con un cliente real, que define los ajustes de interfaz antes del despliegue final.

Prioridad	Acción antes del despliegue final	Tipo	Origen
1. Alta	Aplicar la prueba de usabilidad con cliente real (HT13–HT15) y corregir los hallazgos de severidad alta.	Interfaz	Sección 3
2. Alta	Desplegar en la nube con HTTPS y monitoreo de /api/salud (HT07, RNF-03).	Técnico	DEPLOY.md
3. Alta	Ejecutar 2 o más instancias del backend (o modo clúster) detrás del balanceador para superar el techo de un núcleo.	Técnico	HT12
4. Media	Verificar en producción el efecto de la comparación bcrypt asíncrona (ya aplicada tras HT12) sobre la latencia en picos de inicio de sesión.	Técnico	HT11 / HT12
5. Media	Capacitación por rol de máximo una hora y medición de RNF-01 (traslado sin ayuda).	Puesta en marcha	SCRUM-45
6. Media	Depurar e importar el catálogo real; etiquetar y hacer el conteo general con actas.	Puesta en marcha	SCRUM-42 / 43 / 44
7. Baja	Revisar los términos 'kardex' y 'ajuste' con usuarios de tienda; renombrar si confunden.	Interfaz	Evaluación heurística

Evidencias: reportes HTML y CSV de Locust en docs/evidencias/rendimiento/ (normal.html, estres.html), resultados de pruebas en la terminal (npm run test:*).